

SI 630 Homework 4: Prompt-based NLP

Note: Problems 1, 2, and 3 are answered in the accompanying `hw4.ipynb` notebook. Problem 5 code and plots are also in the notebook; the written trend analysis appears below.

Problem 4

My starting point for prompt design was the most literal interpretation of the task: give the model the context, list the four options, and ask it to pick one. The main design question was how to make the output easy to parse. I initially tried asking the model to repeat the full text of the correct ending, but that made extraction fragile since any paraphrase or truncation would break matching. Switching to a single letter (A, B, C, D) made parsing much more reliable across all models.

From there I explored a few directions. I tried varying the framing of the question—“which continues the context” vs. “pick the most plausible ending”—and found that explicitly labeling the section as “Context:” before the sentence helped models like `flan-t5` orient correctly. For the Chain-of-Thought prompt I added an instruction to reason step by step before committing to a letter. This worked reasonably well for `flan-t5` but caused `Mistral` to generate long outputs where the final letter sometimes appeared far into the response, which initially broke parsing. I also tried a version that asked for JSON output with a “choice” field, but most models either ignored the format or produced malformed JSON, so I dropped it.

The guide on the Huggingface prompting documentation page was useful, particularly the advice to separate options with newlines and to put the instruction *after* the context rather than before. I found that putting the constraint (“answer with a single letter”) at the very end of the prompt, right before the model was expected to continue, produced more consistent outputs than burying it in the middle.

Problem 5 — Trend Analysis

Table ?? shows validation accuracy for all five prompts across all five models.

The results split cleanly along IFT vs. non-IFT lines. `GPT2`, `OPT-2.7b`, and `Gemma-2b` all land at or just above chance (0.25–0.29) regardless of prompt, which confirms that zero-shot classification via generation requires a model that has been instruction fine-tuned. No amount of prompt engineering rescued these models on this task.

Among the IFT models, `flan-t5-large` was the strongest overall, reaching 0.653 with `prompt1` and 0.643 with the in-context learning prompt. Its encoder-decoder structure likely helps here: the encoder reads the full question including all four options, and the decoder only needs to emit a

	<code>prompt1</code>	<code>prompt2_cot</code>	<code>prompt3</code>	<code>prompt_minimal</code>	<code>prompt_icl</code>
GPT2	0.270	0.270	0.270	0.270	0.273
Gemma-2b	0.270	0.270	0.270	0.280	0.273
flan-t5-large	0.653	0.633	0.597	0.427	0.643
OPT-2.7b	0.270	0.270	0.270	0.273	0.293
Mistral-7B	0.587	0.433	0.537	0.287	0.543

Table 1: Validation accuracy (300 examples) for each prompt–model combination. Bold indicates the best overall result.

short label. Mistral-7B was competitive on structured prompts (0.587, 0.543, 0.537 for `prompt1`, `prompt_icl`, `prompt3` respectively) but more sensitive to prompt format. The minimal prompt collapsed Mistral to 0.287, near chance, while flan-t5 degraded more gracefully to 0.427. Chain-of-thought helped flan-t5 slightly less than `prompt1` but still performed well (0.633), whereas for Mistral CoT scored 0.433—lower than simpler formats—because the long reasoning chain made it harder to extract the final answer reliably.

Problem 6

Test set predictions were generated using `google/flan-t5-large` with `prompt1`, which achieved the highest validation accuracy of 0.653 across all prompt–model combinations.

Problem 7

The clearest lesson from this assignment is that instruction fine-tuning is not optional for prompt-based zero-shot classification. Models like GPT2 and OPT were trained to continue text, not follow directions, so giving them a carefully structured multiple-choice question changes nothing about how they generate—they just continue whatever pattern the prompt ends on. This means that before spending time on prompt engineering, the first question should always be whether the base model was trained to respond to instructions at all. Among models that were, format consistency mattered most: as long as the output could be reliably parsed, moderate differences in phrasing had less impact than I expected. The bigger risk was over-engineering prompts that confused the model or pushed the answer token far from the start of the output.

Applying this to non-multiple-choice tasks would require more thought about output structure. For open-ended classification or extraction, asking the model to output a specific keyword or a short JSON field is probably more robust than asking for a letter, since there is no natural “A/B/C/D” equivalent. Chain-of-thought seems most useful when the task genuinely requires multi-step reasoning—for HellaSwag, which is largely about plausibility rather than logic chains, CoT did not help much and sometimes hurt by producing longer outputs. For tasks like entailment or complex relation extraction, CoT would likely add more value.

Advice for prompt design:

1. **Fix the output format first.** Decide exactly what token or string you want the model to produce and design the prompt around making that token likely. Everything else is secondary to having a parseable output.
2. **Put the constraint at the end.** Instructions like “answer with a single letter” work better

when they appear immediately before the model's turn, not buried in the middle of the prompt where they can be forgotten.

3. **Test on a small slice before scaling.** Running five examples manually and inspecting raw model output is faster than running the full dataset and debugging aggregate accuracy. Many parsing bugs and formatting failures are obvious from a handful of examples.